Benjamin Garcia

(626) 246-4778 | bentgarcia05@gmail.com | GitHub | LinkedIn | Portfolio

Education

University of California, Los Angeles — Los Angeles, CA
Bachelor of Science, Computer Science — *Expected Graduation: Dec. 2027*

Technical Skills

Languages: TypeScript, Python, Java, C++, SQL, JavaScript, HTML/CSS
Frameworks & Technologies: Next.js, React, Node.js, FastAPI, GraphQL, Tailwind CSS, Prisma, Zod, React Flow, Temporal
Databases & Infrastructure: PostgreSQL, MongoDB, Redis, SQLite, Supabase, Docker, Git, Linux/Unix

Experience

Software Engineer Intern — Jun. 2026 – Present
Lindy — *San Francisco, CA*

- **Hardened multi-account Email and Meetings infrastructure** serving **51,127 multi-account users**, separating OAuth auth, per-account settings, and agent automation state so account removal, reconnect, sync, and opt-out flows could not silently re-enable disabled inbox or calendar behavior.
- **Built unified Integrations account-management flows across web and mobile**, including add, reconnect, delete, rename, share, provider parity, and feature-flagged Connections fallback; hardened auth handling around workspace permissions, pending auth requests, provider/method matching, and stale persisted references.
- **Improved scheduling and follow-up draft quality across Gmail and Outlook** by wiring user drafting instructions, learned email style, holiday exclusions, account-aware meeting classification, and email rendering fixes into production draft-generation flows, contributing to scheduling draft user-requested edit rate falling from **14.9% to 3.2%** and draft-alert pushback falling from **6.7% to 3.9%**.

Undergraduate Researcher — Dec. 2025 – Present
UCLA Human–Computer Interaction Research Lab — *Los Angeles, CA*

- Building **HI-COS/MARS**, a human-in-the-loop AI research platform that transforms retrieved scientific literature into evidence-grounded agent perspectives, structured debate, and refined research hypotheses.
- Implemented a React Flow hypothesis workspace backed by FastAPI event streams, visualizing **8** HI-COS event variants and **19** MARS workflow events across agent reasoning, critique, refinement, and persistence.
- Developed a **5-stage**, **20-step** literature-grounded workflow that retrieves Semantic Scholar papers, clusters evidence by embeddings, synthesizes citation-grounded researcher personas, and surfaces provenance in inspectable UI panels.

Software Engineer Intern — Mar. 2025 – Sep. 2025
Todd Agriscience — *Los Angeles, CA*

- Delivered **AI-driven crop insight dashboards** for **5–10** clients, integrating full-stack Next.js interfaces with Palantir Foundry and AWS Lambda.
- Developed data pipelines visualizing **20+** agricultural streams across soil, irrigation, weather, and yield data to support real-time farm management decisions.

Software Engineer Intern — May 2025 – Jun. 2025
TensorStax — *San Francisco, CA*

- Engineered **real-time agentic UIs** with Next.js, Redux, and WebSockets, supporting **100+** concurrent users with **sub-100ms** latency.
- Built **secure credential-submission workflows** with Python, Redis, and HashiCorp Vault, supporting authentication across **50+** enterprise data sources.

Projects

GitProof | *Next.js, TypeScript, PostgreSQL, Prisma, Gemini API, GitHub GraphQL API* — Dec. 2025 – Present

- Built a full-stack developer portfolio platform used by **10** users to transform GitHub activity into recruiter-ready profiles, processing **100+** public repositories through a type-safe GitHub GraphQL, PostgreSQL, and Prisma pipeline.
- Designed a **3-factor** weighted impact scoring algorithm combining logarithmic popularity, recency decay across **5** time tiers, and project maturity signals into a **0–50** portfolio score.
- Built an **AI-powered insights system** using Google Gemini 2.5 Flash to generate project descriptions, bios, and portfolio feedback from repository metadata.

Logit | *Next.js, TypeScript, PostgreSQL, Prisma, React, Recharts, Anthropic API* — Mar. 2026 – Present

- Built a full-stack workout journal piloted by **5** lifters, supporting workout logging, split planning, analytics, public profiles, and **AI-assisted split generation** across an **11-model** PostgreSQL schema.
- Engineered **transaction-safe workout and split mutations** across **12** API route files, combining validation, weight normalization, derived read models, cache invalidation, and trusted-origin protections.
- Wrote **23** TypeScript test files covering **76** cases across workout math, scheduling, split rules, AI draft validation, payload normalization, read-model sync, and export formatting.